

You should get *[OK]* for all the tests. If you get a *[Failed]*, then it's a fatal error; if you start RTLlinux, it might crash the kernel. Something has gone wrong during the compilation and installation of the kernel. In this case, I suggest you try to build the kernel again. You will find some very good FAQ at the URL given below, which may be of help: <http://bit.ly/d1V7DO>

If all tests return *OK*, then run the following commands:

```
rtllinux status
rtllinux start
```

These will install all the modules present in the */usr/src/rtllinux/modules* directory. Our own module will use only the symbols that are present in these modules.

Let's get our hands dirty

Once RTLlinux is set up, it's ready to run a test module. To understand it better, there are a lot of *examples* provided along with the RTLlinux patch, which you will find in the *examples* sub-directory. Let's say 'Hello' to RTLlinux using the hello module.

1. To compile, go to the examples directory, and execute the *make* command. The *Makefile* that should be already there will be used to create the object file.
2. Now run the command *insmod filename.o*, where *filename* is the module name you want to load into the running kernel. For example, *insmod hello.o*.
3. To view a list of loaded modules, use the *lsmod* command.
4. To view the messages output by the newly loaded module, use the *dmesg* command.

A code fragment decoded

This program can be found in *examples/hello*. The aim of the module is to execute twice a second; during each execution, it will print the message "I'm here, my arg is 0".

```
#include <rtl.h>
#include <time.h>
#include <pthread.h>
pthread_t thread;
void * start_routine(void *arg) {
    struct sched_param p;
    p.sched_priority = 1;
    pthread_setschedparam (pthread_self(), SCHED_FIFO, &p);
    pthread_make_periodic_np (pthread_self(), gethrtime(),
5000000000);
    while (1) {
        pthread_wait_np();
        rtl_printf("I'm here; my arg is %x\n", (unsigned) arg);
    }
    return 0;
}
int init_module(void) {
```

```
    return pthread_create (&thread, NULL, start_routine, 0);
}
void cleanup_module(void) {
    pthread_cancel (thread);
    pthread_join (thread, NULL);
}
```

Now let me explain what each part of this code does.

1) *init_module()*

The module is initialised using this function. It executes *pthread_create* to create a thread, and returns its success status. On successful execution of *pthread_create*, the *start_routine* function is called, with 0 as its argument.

2) *start_routine()*

This comprises three components: initialisation, run-time and termination.

Initialisation

Executed when the code is run the first time after loading, the section of code written before the *while* loop tells the scheduler to assign this thread scheduling a priority of one, using *p.sched_priority*. It then sets the scheduler's behaviour to be *SCHED_FIFO* for all subsequent executions, using *pthread_setschedparam*. Finally, it calls the function *pthread_make_periodic_np* to run this thread after the given time period of 500 milliseconds.

Run-time

The block of code inside the *while* loop blocks all further execution of the thread using *pthread_wait_np()*. Once the thread is called again, after 500 milliseconds, it executes the rest of the code inside the *while* loop. However, since this is an infinite loop, with no exit condition, unless the module is unloaded using the *rmmod* command, it will keep executing the thread every 500 milliseconds.

Termination

The function **cleanup_module()* is called when the module is unloaded using 'rmmod' command.

```
*# rmmod hello*
```

Then it calls *pthread_cancel()* to cancel the thread and *pthread_join()* to wait for the thread termination. This helps in freeing up any resources being used by the module. **END** 

By: Manoj Kumar

The author leads a team in Linux Kernel Programming, Linux Administration, Cluster Computing, Embedded Systems and QT/GTK programming on Linux. You can reach him at manoj@nxtsolutions.com and see the latest discussions on his group, http://tech.groups.yahoo.com/group/FutureOS_Linux/.